



AUTOMATED WEATHER REPORTER

Python Mini Project Report

Project Type: Beginner Python GUI + API Project

Technology: Python, Tkinter, Requests

API: OpenWeatherMap Forecast API

NAME – ISHA GUPTA
BRANCH – CSE(AI ML)
REG. NO. – 26BAI11037

Table of Content

- 1. Introduction**
- 2. Problem Statement**
- 3. Functional Requirements**
- 4. Non-Functional Requirements**
- 5. System Architecture**
- 6. Design Diagrams**
- 7. Design Decisions & Rationale**
- 8. Implementation Details**
- 9. Screenshots / Results**
- 10. Testing Approach**
- 11. Challenges Faced**
- 12. Conclusion Additional pages expand the code, setup, API flow, testing, diagrams, and screenshot explanations.**

Introduction

The Automated Weather Reporter is a beginner-friendly Python desktop application that retrieves weather information for a city and displays two simple values: temperature and chance of rain. The goal is to demonstrate how a small Python program can communicate with an external web API and present the returned data through a graphical interface. The application uses Tkinter for the graphical user interface and the Requests library for HTTP communication. openweathermap provides the weather forecast data. The user enters a city name and clicks a button; the application sends the city and API key to the service, receives JSON data, extracts the required values, and displays them. This project is intentionally limited in scope so that the underlying programming concepts remain easy to understand and explain during a project evaluation.

Problem Statement

A user may need a quick answer to two basic weather questions: “What is the temperature?” and “What is the chance of rain?” A weather application containing many unrelated measurements can make a beginner project unnecessarily complex.

Problem: Create a simple application that accepts a city name and retrieves useful weather information automatically from an online service.

Objective: Build a Python GUI that displays temperature in Celsius and precipitation probability as a percentage.

Scope: The current version does not store data, require a login, or provide a long multi-day dashboard. It focuses on a single simple weather lookup.

Functional Requirements

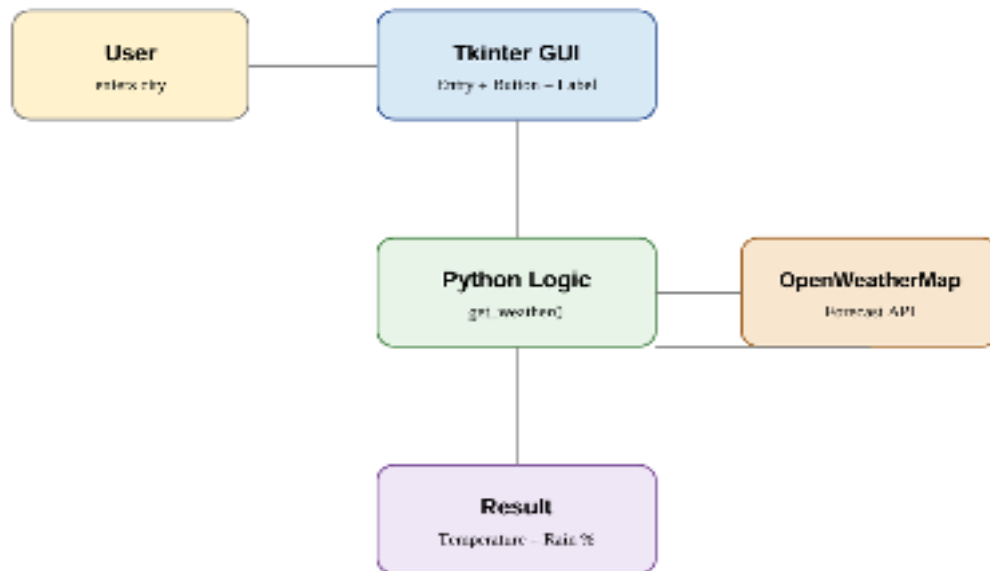
ID	REQUIREMENT	DESCRIPTION
1	CITY INPUT	User can type a city name.
2	WEATHER REQUEST	Program sends the city and API key to Openweathermap.
3	TEMPERATURE	Program extracts and displays temperature in °C.
4	Rain probability	Program extracts precipitation probability and converts it to %
5	Result display	GUI updates a label with both values
6	Basic error handling	Invalid requests do not produce a normal weather result.

Non-Functional Requirements

- Usability: The interface should be understandable without technical knowledge.
- Performance: The application should make a single lightweight API request per lookup.
- Reliability: Invalid cities and unsuccessful responses should be handled.
- Maintainability: Code should use clear names and simple functions.
- Portability: The program should work wherever a compatible Python/Tkinter environment is available.
- Security: The API key should not be committed to a public repository.

A safer later version should load it from an environment variable.

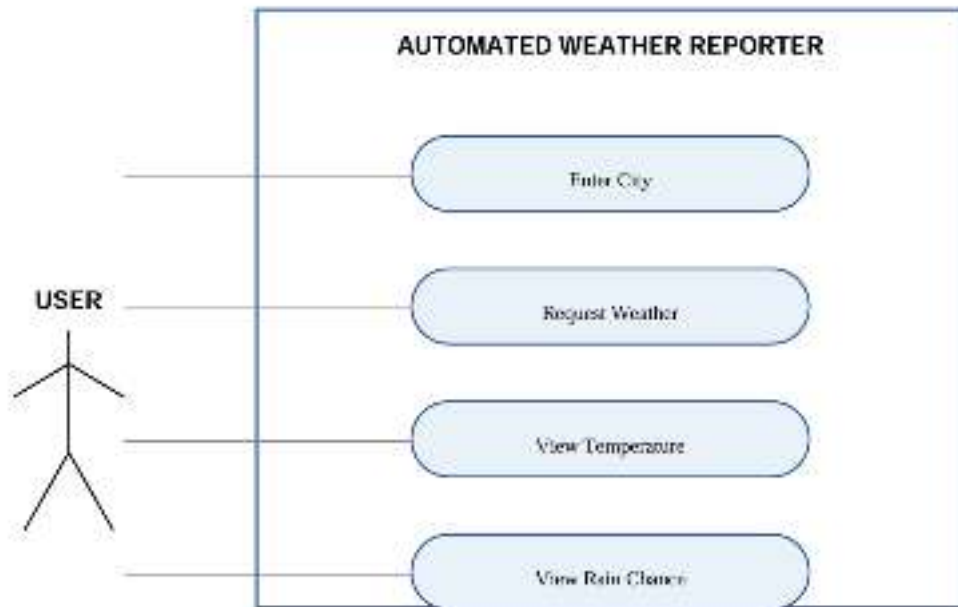
System Architecture



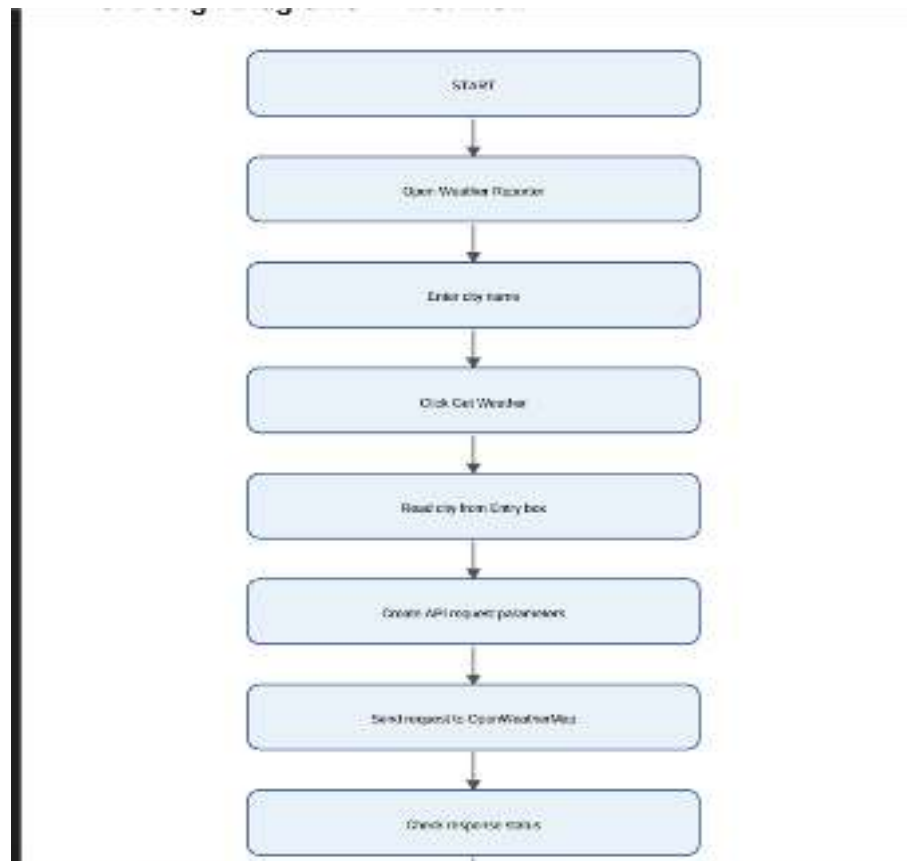
The system has four logical layers: user interface, Python application logic, external API communication, and result presentation.

Design Diagrams — Use Case

The use case diagram identifies the user as the main actor. The user enters a city and requests weather information. The system then obtains and presents temperature and rain probability.



Design Diagrams — Workflow



Design Diagrams — Sequence Diagram

sequence of iterations :

Step	Sender	Receiver	Action
1	User	GUI	Enters city
2	User	GUI	Clicks Get Weather
3	GUI	Python function	Calls get_weather()
4	Python	OpenWeatherMap	Sends HTTP request
5	OpenWeatherMap	Python	Returns JSON
6	Python	GUI	Updates result_label
7	GUI	User	Shows temperature and rain probability

Design Diagrams — Class/Component View

GUI Component: creates the window, label, input box, button, and result label.

Weather Function: `get_weather()` reads input, creates the request, checks the response, extracts values, and updates the GUI.

Requests Component: performs HTTP communication.

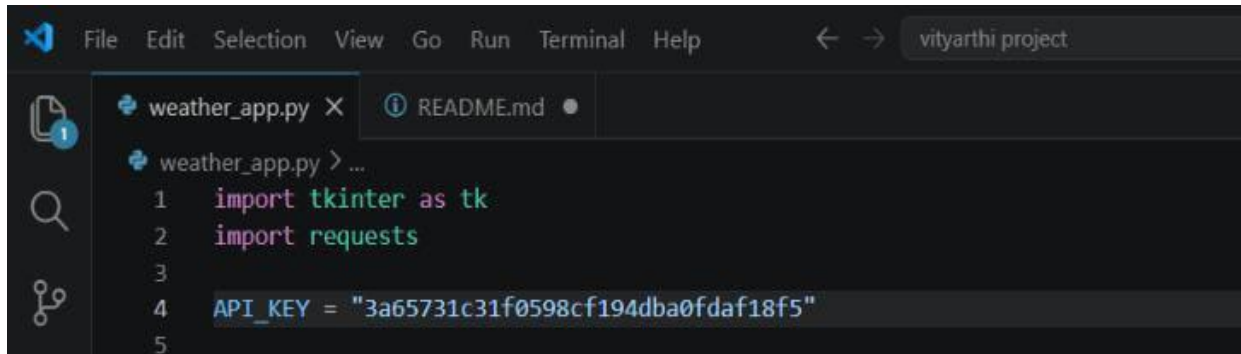
OpenWeatherMap API: external data source.

Storage: none in the current version.

Design Decisions & Rationale

- **Tkinter:** chosen for a simple desktop GUI with minimal additional setup.
- **Requests:** provides a readable interface for HTTP GET requests.
- **OpenWeatherMap:** supplies forecast data through a web API.
- **Celsius:** selected through the metric unit parameter.
- **Minimal output:** temperature and rain probability are sufficient for the stated project objective.
- **No database:** the application does not need to save previous searches.
- **Single function:** `get_weather()` keeps the beginner implementation easy to trace.

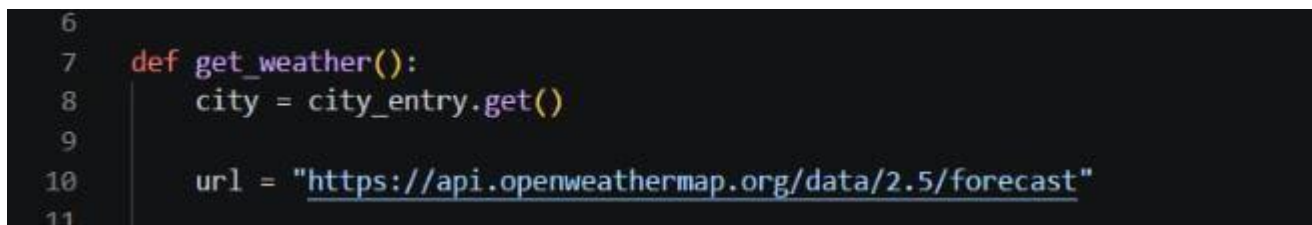
Implementation Details — Imports and Configuration



```
File Edit Selection View Go Run Terminal Help
weather_app.py X README.md
weather_app.py > ...
1 import tkinter as tk
2 import requests
3
4 API_KEY = "3a65731c31f0598cf194dba0fdaf18f5"
5
```

tkinter provides GUI widgets. **requests** provides the HTTP request function. The API key identifies the application to the weather service.

Implementation Details — Reading Input



```
6
7 def get_weather():
8     city = city_entry.get()
9
10     url = "https://api.openweathermap.org/data/2.5/forecast"
11
```

The function is called when the button is clicked. `city_entry` is the Tkinter Entry widget. The `get()` method reads whatever text the user entered. Example: if the user enters **Bhopal**, the variable `city` contains the string "Bhopal".

Implementation Details — API Request

```
11
12     params = {
13         "q": city,
14         "appid": API_KEY,
15         "units": "metric"
16     }
17
18     response = requests.get(url, params=params)
19     data = response.json()
```

The URL identifies the forecast endpoint.

q is the city, appid is the API key, and units=metric requests Celsius-based values.
requests.get() sends these parameters to the service.

Implementation Details — Response and Error Check

```
19     data = response.json()
20
21     if response.status_code != 200:
22         result_label.config(text="City not found!")
23         return
24
```

The response is converted from JSON to Python data. The status code is checked before attempting to extract normal weather values. A non-200 response is treated as an unsuccessful request in this beginner version. A production version should distinguish invalid-city responses, rate limits, authentication errors, and network failures more precisely.

Implementation Details — Temperature

```
25 # Temperature
26 temperature = data["list"][0]["main"]["temp"]
```

The forecast endpoint returns a list of forecast entries. The first entry is used by this simple application. The program enters the main object and reads the temp field. Because the request uses units=metric, the temperature value is presented in Celsius.

Implementation Details — Chance of Rain

```
27
28 # Chance of rain
29 rain_chance = data["list"][0]["pop"] * 100
30
```

The precipitation probability value is represented as a decimal fraction. Multiplying by 100 converts it into a percentage for display. For example, a value of 0.60 becomes 60%. The formatted output uses :.0f so the GUI shows a whole-number percentage.

Implementation Details – GUI

```
35
36
37 # Create window
38 window = tk.Tk()
39 window.title("Weather Reporter")
40 window.geometry("400x300")
41
```

Tk() creates the main window. title() sets its name and geometry() sets the initial size.

```
42
43 # Heading
44 heading = tk.Label(
45     window,
46     text="Weather Reporter",
47     font=("Arial", 22, "bold")
48 )
49 heading.pack(pady=20)
50
51
```

A Label displays text. pack() places the widget in the window.

Implementation Details — Input, Button and Result

```
59
60 # City input
61 city_entry = tk.Entry(
62     window,
63     width=25,
64     font=("Arial", 14)
65 )
66 city_entry.pack(pady=10)
67
68
69 # Button
70 weather_button = tk.Button(
71     window,
```

Entry provides a text box. Button creates the clickable control.
command=get_weather connects the button to the weather function.

```
78 # Result
79 result_label = tk.Label(
80     window,
81     text="Temperature: --\nChance of Rain: --",
82     font=("Arial", 16)
83 )
84 result_label.pack(pady=20)
85
```

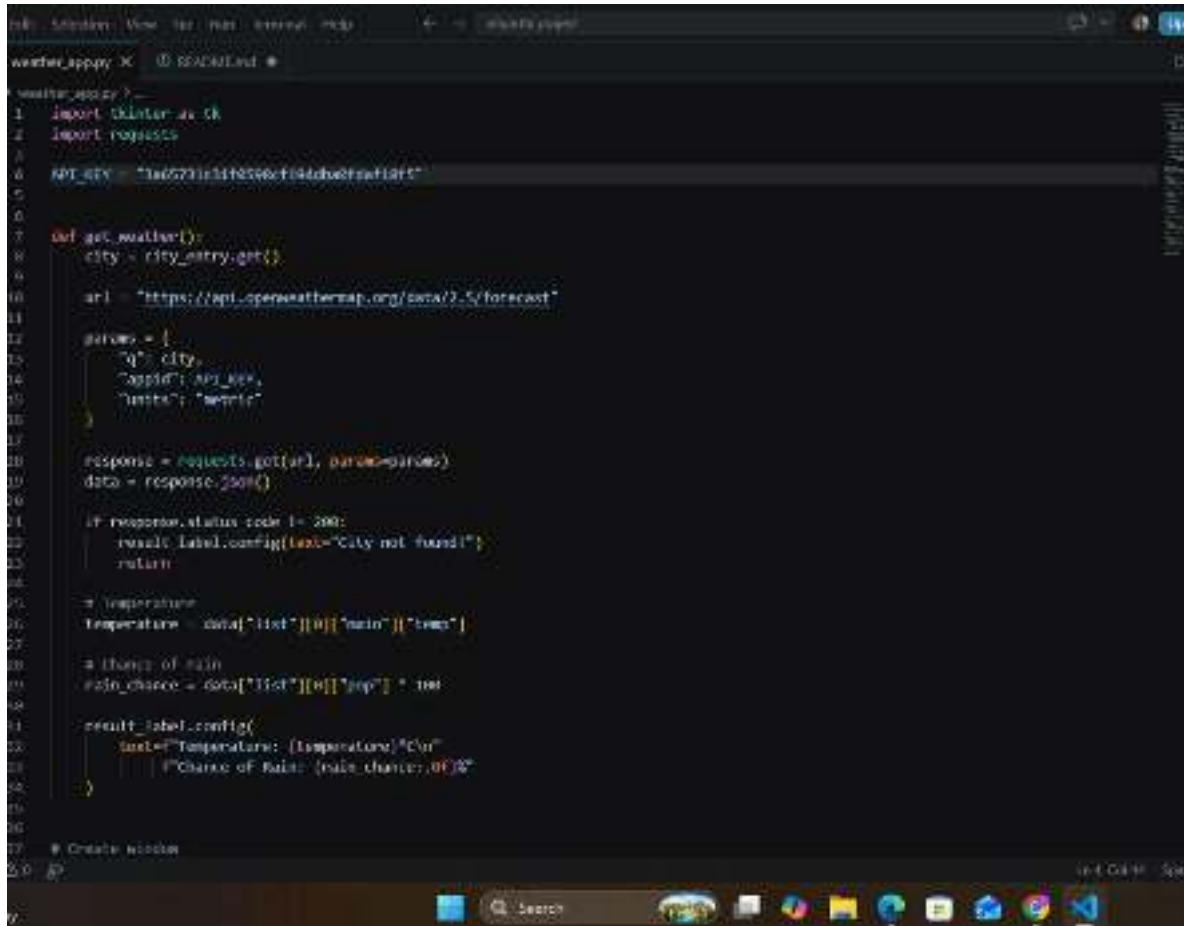
The result label initially shows placeholders and is updated after the API request completes.

Implementation Details — Final Display and Main Loop

```
30
31     result_label.config(
32         text=f"Temperature: {temperature}°C\n"
33         | f"Chance of Rain: {rain_chance:.0f}%"
34     )
35
```

config() changes the label text after data is received. The newline character creates two output lines. mainloop() keeps the GUI running and waits for user actions

CODE SCREENSHOT-1



```
weather_app.py X ID READ-ONLY *
weather_app.py? ...
1 import tkinter as tk
2 import requests
3
4 API_KEY = "1a65771c1c1f0556c6f44d4b0e7a1a1f"
5
6
7 def get_weather():
8     city = city_entry.get()
9
10    url = "https://api.openweathermap.org/data/2.5/forecast"
11
12    params = {
13        "q": city,
14        "appid": API_KEY,
15        "units": "metric"
16    }
17
18    response = requests.get(url, params=params)
19    data = response.json()
20
21    if response.status_code != 200:
22        result_label.config(text="City not found!")
23        return
24
25    # Temperature
26    temperature = data["list"][0]["main"]["temp"]
27
28    # Chance of rain
29    rain_chance = data["list"][0]["pop"] * 100
30
31    result_label.config(
32        text=f"Temperature: {temperature}°C\n"
33            f"Chance of Rain: {rain_chance}%")
34
35
36
37 # Create window
```

This screenshot shows the API and data-processing portion of the code. It imports Tkinter and Requests, defines the API key variable, creates the forecast URL and parameters, sends the request, converts the response to JSON, extracts temperature and precipitation probability, and updates the result label.

CODE SCREENSHOT -2

```
weather_app.py X README.md *
weather_app.py > ...
35
36
37 # Create window
38 window = tk.Tk()
39 window.title("Weather Reporter")
40 window.geometry("400x300")
41
42
43 # heading
44 heading = tk.Label(
45     window,
46     text="Weather Reporter",
47     font=("Arial", 22, "bold")
48 )
49 heading.pack(pady=20)
50
51
52 # City label
53 city_label = tk.Label(
54     window,
55     text="Enter City:"
56 )
57 city_label.pack()
58
59
60 # City input
61 city_entry = tk.Entry(
62     window,
63     width=25,
64     font=("Arial", 14)
65 )
66 city_entry.pack(pady=10)
67
68
69 # Button
70 weather_button = tk.Button(
71     window,
```

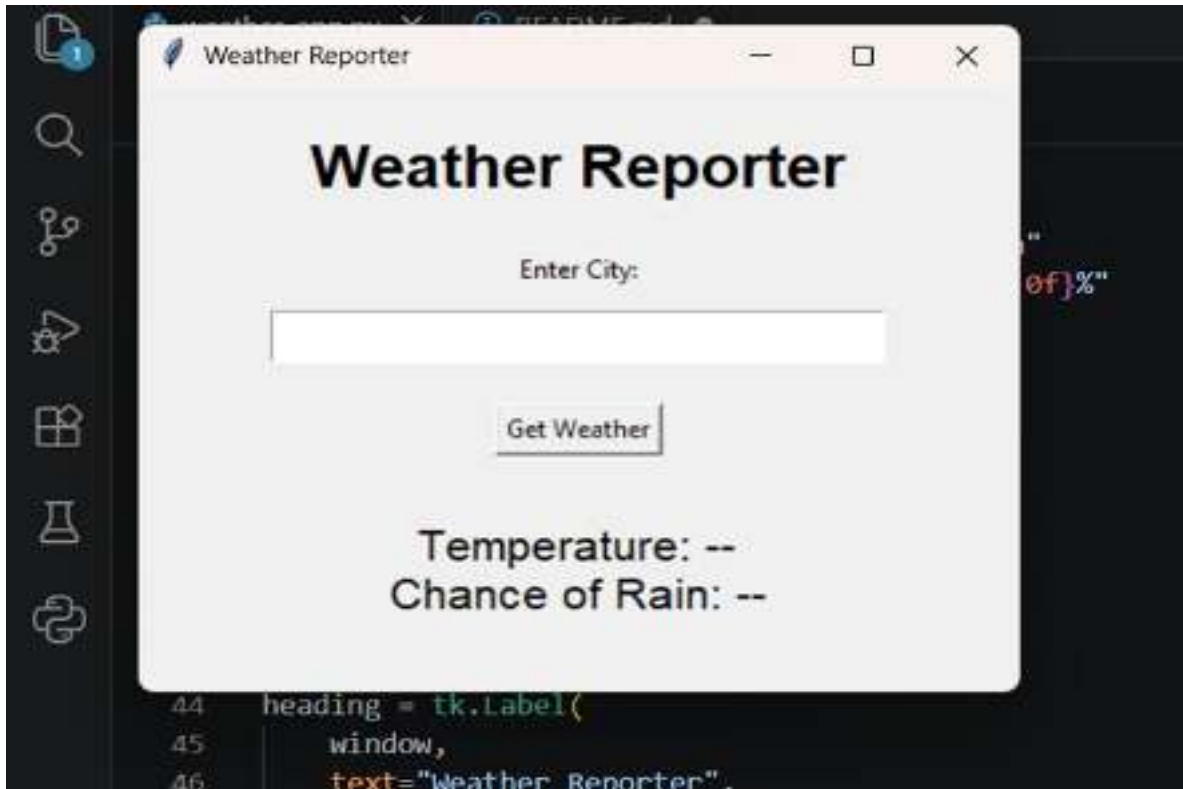
CODE SCREENSHOT 3

```
weather_app.py X  @ README.md ●
weather_app.py > -
28
29 # City input
30 city_entry = tk.Entry(
31     window,
32     width=25,
33     font=("Arial", 14)
34 )
35 city_entry.pack(pady=10)
36
37
38 # Button
39 weather_button = tk.Button(
40     window,
41     text="Get Weather",
42     command=get_weather
43 )
44 weather_button.pack(pady=10)
45
46
47 # Result
48 result_label = tk.Label(
49     window,
50     text="Temperature: --\nChance of Rain: --",
51     font=("Arial", 16)
52 )
53 result_label.pack(pady=20)
54
55
56 # Start application
57 window.mainloop()
```

Explanation: This screenshot shows the city input, Get Weather button, and result label. The command=get_weather setting connects the button click to the function that performs the weather lookup.

Screenshots / Results — Expected Output

After the program is run, the user sees a small window with the title Weather Reporter



THEN ENTER THE CITY NAME YOU WANT TO GET INFORMATION ABOUT:

OUTPUT



Testing Approach

Testing should be repeated after any code modification.

Test ID	Test	Expected Result
T01	Valid city name	Weather values displayed.
T02	Another valid city	Correct city request processed.
T03	Invalid city	Error message displayed.
T04	Blank city	Application should reject or prompt for input.
T05	No internet	Application should handle request failure.
T06	Invalid API key	API error should be handled without crashing.
T07	Button click	get_weather() executes.
T08	Result formatting	Temperature and rain percentage appear on separate lines.

Testing should be repeated after any code modification.

Testing — Sample Test Procedure

Test 1: Valid city Enter a known city, such as Delhi, and click Get Weather. Confirm that both temperature and rain probability appear.

Test 2: Invalid city Enter an intentionally invalid city name. Confirm that the application does not attempt to display normal weather values.

Test 3: Empty input Leave the Entry field empty and click the button. Add validation if the current code does not already handle it.

Test 4: Network/API failure Run without internet or with an invalid API key and verify that the application handles the unsuccessful request.

Challenges Faced

- Installing the Requests library and understanding why it is needed.
- Creating and activating an OpenWeatherMap API key.
- Understanding the structure of JSON returned by the API.
- Finding the correct temperature and precipitation fields.
- Connecting a GUI button to a Python function.
- Understanding HTTP status codes and basic error handling.
- Keeping the project simple enough for a beginner while still meeting the API requirement.

Conclusion & Future Scope

The Automated Weather Reporter successfully demonstrates a complete beginner project pipeline: GUI input, API request, JSON processing, and GUI output.

The project is small enough to understand line by line while still showing how real applications use external services.

Future improvements: • Use environment variables for the API key.

- Add an explicit blank-input validation message.
- Handle different API error codes separately.
- Add weather condition icons.
- Show a multi-day forecast.
- Add automatic location detection.
- Store recent searches if persistent storage becomes a requirement.

The current version intentionally avoids these additions so the core concepts remain clear.